Section 16 : Polymorphism

Information

- Title: `Polymorphism`
- Authors: `Etienne KOA`

Aperçu

Polymorphisme et héritage

- Qu'est-ce que le polymorphisme ?
- Utilisation de pointeurs de classe de base
- Liaison statique ou dynamique
- Fonctions virtuelles
- Destructeurs virtuels
- Les spécificateurs de remplacement et finaux
- Utilisation de références de classe de base
- Fonctions virtuelles pures et classes abstraites
- Classes abstraites comme interfaces

Qu'est-ce que le polymorphisme

- Fondamental de la programmation orientée objet
- Polymorphisme
  - Au moment de la **compilation/liaison anticipée/liaison statique**
  - **Exécution / liaison tardive / liaison dynamique**
- Polymorphisme d'exécution
  - Être capable d'attribuer une signification différente à la même fonction au moment de l'exécution
- Nous permet de programmer de manière plus abstraite
  - Pensez général plutôt que spécifique
  - Laissez `C++` déterminer quelle fonction appeler au moment de l'exécution
- Ce n'est pas la valeur par défaut en `C++`, le polymorphisme d'exécution est obtenu via
  - Héritage
  - Pointeurs ou références de classe de base
  - Fonctions virtuelles

Types de polymorphisme en C++ ?

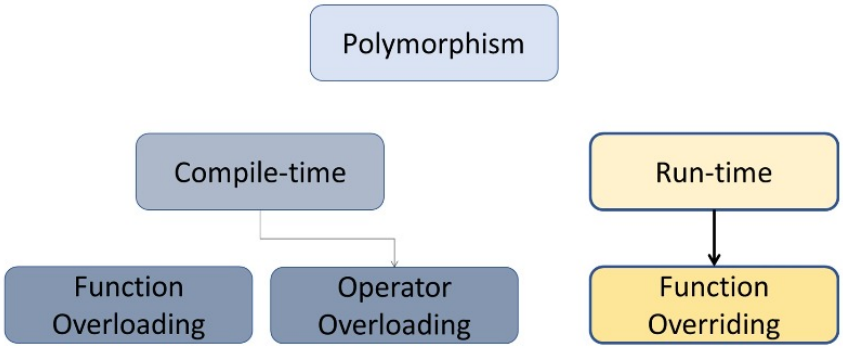
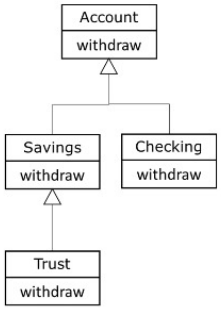


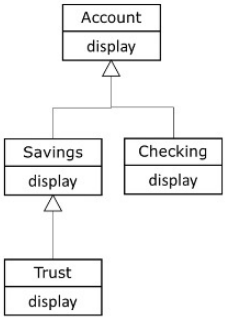
Figure 1: Polymorphism

Un exemple non polymorphe - Static Binding

```
Account a;
a.withdraw(1000);    // Account::withdraw()

Savings b;
b.withdraw(1000);    // Savings::withdraw()

Checking c;
c.withdraw(1000);    // Checking::withdraw()

Trust d;
d.withdraw(1000);    // Trust::withdraw()


Account *p = new Trust();
P->withdraw(1000);   // Account::withdraw()
                     // should be
                     // Trust::withdraw()
```



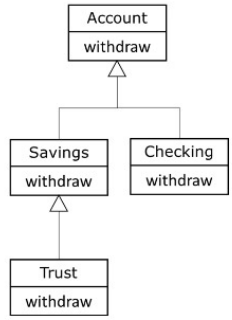

```
void display_account(const Account &acc)  {
   acc.display();
   // will always use Account::display
}

Account a;
display_account(a);

Savings b;
display_account(b);

Checking c;
display_account(c);

Trust d;
display_account(d);
```



Un exemple polymorphe - Dynamic Binding

A polymorphic example – Dynamic Binding

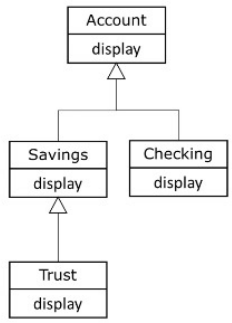
```
Account a;
a.withdraw(1000);    // Account::withdraw()

Savings b;
b.withdraw(1000);    // Savings::withdraw()

Checking c;
c.withdraw(1000);    // Checking::withdraw()

Trust d;
d.withdraw(1000);    // Trust::withdraw()


Account *p = new Trust();
P->withdraw(1000);   // Trust::withdraw()
```



`withdraw` method is virtual in Account

A polymorphic example – Dynamic Binding

```
void display_account(const Account &acc)
{
   acc.display();
   // will always call the display method
   // depending on the object's type
   // at RUN-TIME!
}

Account a;
display_account(a);

Savings b;
display_account(b);

Checking c;
display_account(c);

Trust d;
display_account(d);
```

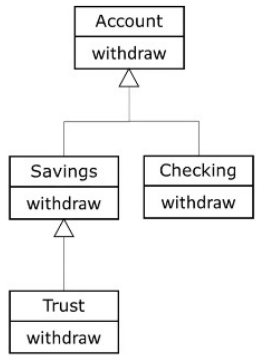


`display` method is virtual in Account

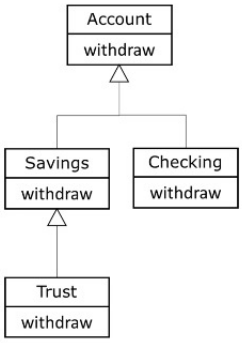
Utilisation d'un pointeur de classe de base

Utilisation d'un pointeur de classe de base

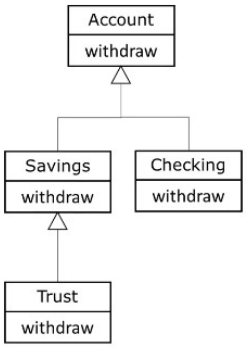
```
Account *p1 = new Account();
Account *p2 = new Savings();
Account *p3 = new Checking();
Account *p4 = new Trust();

p1->withdraw(1000);        //Account::withdraw
p2->withdraw(1000);        //Savings::withdraw
p3->withdraw(1000);        //Checking::withdraw
p4->withdraw(1000);        //Trust::withdraw

// delete the pointers
```



```
Account *p1 = new Account();
Account *p2 = new Savings();
Account *p3 = new Checking();
Account *p4 = new Trust();

Account *array [] = {p1,p2,p3,p4};

for (auto i=0; i<4; ++i)
   array[i]->withdraw(1000);
```




```
Account *p1 = new Account();
Account *p2 = new Savings();
Account *p3 = new Checking();
Account *p4 = new Trust();

vector<Account *> accounts
      {p1, p2, p3, p4 };

for (auto acc_ptr: accounts)
   acc_ptr->withdraw(1000);

// delete the pointers
```



Fonctions virtuelles

Polymorphisme

- Fonctions virtuelles
  - Les fonctions redéfinies sont liées statiquement
  - Les fonctions remplacées sont liées dynamiquement
  - Les fonctions virtuelles sont remplacées
  - Permettez-nous de traiter tous les objets généralement comme des objets de la classe Base

Déclaration de fonctions virtuelles + Déclarez la fonction que vous souhaitez remplacer comme virtuelle dans la classe Base + Les fonctions virtuelles sont virtuelles tout au long de la hiérarchie à partir de ce point + Polymorphisme dynamique uniquement via le pointeur ou la référence de classe `Account`

- Remplacer la fonction dans les classes dérivées
- La signature de la fonction et le type de retour doivent correspondre EXACTEMENT
- Mot-clé virtuel non obligatoire mais constitue une bonne pratique
- Si vous ne fournissez pas de version remplacée, elle est héritée de sa classe de base

```
class Account {
public:
    virtual void withdraw(double amount);
    . . .
};
```

Figure 2: Polymorphism

```
class Checking : public Account {
public:
    virtual void withdraw(double amount);
    . . .
};
```

Figure 3: Polymorphism

Destructeurs virtuels

Polymorphisme

- Destructeurs virtuels
  - Des problèmes peuvent survenir lorsque nous détruisons des objets polymorphes
  - Si une classe dérivée est détruite en supprimant son stockage via le pointeur de classe de base et la classe un destructeur non virtuel Alors le comportement n'est pas défini dans le standard `C++`.
  - Les objets dérivés doivent être détruits dans le bon ordre en commençant par le bon destructeur.
- Solution/Règle :
  - Si une classe a des fonctions virtuelles
  - TOUJOURS fournir un destructeur virtuel public
  - Si le destructeur de classe de base est virtuel alors tous les destructeurs de classes dérivées sont également virtuels

Utilisation du spécificateur de remplacement

Le spécificateur de remplacement

- Nous pouvons remplacer les fonctions virtuelles de la classe Base
- La signature et le retour des fonctions doivent être EXACTEMENT les mêmes

```
class Account {
public:
    virtual void withdraw(double amount);
    virtual ~Account();
    . . .
};
```

Figure 4: Polymorphism

- S'ils sont différents, alors nous avons une redéfinition qui n'est PAS prioritaire
- La redéfinition est liée statiquement
- Le remplacement est lié dynamiquement
- `C++11` fournit un spécificateur de remplacement pour que le compilateur assure le remplacement
- Le spécificateur de remplacement

```
class Base {
public:
    virtual void say_hello() const {
        std::cout << "Hello - I'm a Base class object" << std::endl;
    }
    virtual ~Base() {}
};

class Derived: public Base {
public:
    virtual void say_hello()   {   // Notice I forgot the const - NOT OVERRIDING
        std::cout << "Hello - I'm a Derived class object" << std::endl;
    }
    virtual ~Derived() {}
};
```

```
Base:

    virtual void say_hello() const;


Derived:

    virtual void say_hello();
```

- Pas ce à quoi nous nous attendions
- Les signatures de la méthode `say_hello` sont différentes
- Ainsi dérivé `redéfinit` `say_hello` au lieu de le remplacer !

```
Base *p1 = new Base();
p1->say_hello();          // "Hello - I'm a Base class object"

Base *p2 = new Derived();
p2->say_hello();          // "Hello - I'm a Base class object"
```

Figure 5: Polymorphism

```
class Base {
public:
    virtual void say_hello() const {
        std::cout << "Hello - I'm a Base class object" << std::endl;
    }
    virtual ~Base() {}
};

class Derived: public Base {
public:
    virtual void say_hello() override  {   // Produces compiler error
                                           // Error: marked override but does not override
        std::cout << "Hello - I'm a Derived class object" << std::endl;
    }
    virtual ~Derived() {}
};
```

Figure 6: Polymorphism

Utilisation du spécificateur final

Le spécificateur `final`

- `C++11` fournit le spécificateur final
  - Lorsqu'il est utilisé au niveau de la classe
  - Empêche une classe d'être dérivée de
  - Lorsqu'il est utilisé au niveau de la méthode
  - Empêche la méthode virtuelle d'être remplacée dans les classes dérivées
- classe `final`
- méthode `final`

Utilisation des références de classe de base

- Nous pouvons également utiliser des références de classe Base avec un polymorphisme dynamique
- Utile lorsque nous passons des objets à des fonctions qui attendent une référence de classe de base

```
class My_class final {
    . . .
};

class Derived final: public Base {
    . . .
};
```

Figure 7: Polymorphism

```
class A {
public:
   virtual void do_something();
};

class B: public A {
   virtual void do_something() final;   // prevent futher overriding
};

class C: public B {
   virtual void do_something();         // COMPILER ERROR - Can't override
};
```

Figure 8: Polymorphism

```
Account a;
Account &ref = a;
ref.withdraw(1000); // Account::withdraw

Trust t;
Account &ref1 = t;
ref1.withdraw(1000);// Trust::withdraw
```

```
void do_withdraw(Account &account, double amount) {
   account.withdraw(amount);
}

Account a;
do_withdraw(a, 1000);  // Account::withdraw

Trust t;
do_withdraw(t, 1000);  // Trust::withdraw
```

Fonctions virtuelles pures et classes abstraites

- Cours abstrait
  - Impossible d'instancier des objets
  - Ces classes sont utilisées comme classes de base dans les hiérarchies d'héritage
  - Souvent appelées classes de base abstraites
- Cours de béton
  - Utilisé pour instancier des objets à partir de
  - Toutes leurs fonctions membres sont définies
    - `Checking Account, Savings Account`
    - `Facultaty, Staff`
    - `Enemy, Level Boss`
- Classe de base abstraite
  - Trop générique pour créer des objets à partir de
    - `Shape, Employee, Account, Player`
  - Serveur en tant que parent des classes dérivées pouvant avoir des objets
  - Contient au moins une fonction virtuelle pure
- Fonction virtuelle pure

  - Utilisé pour créer une classe « abstraite »
  - Spécifié avec `=0` dans sa déclaration

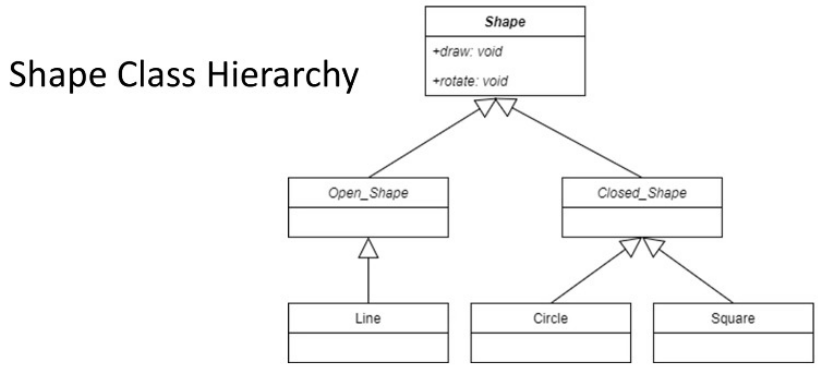
```
virtual void function() = 0; // pure virtual function
```

  - Ne fournissent généralement pas d'implémentations

- Fonction virtuelle pure
  - Les classes dérivées DOIVENT remplacer la classe de base
  - Si la classe Derived ne remplace pas, alors la classe Derived est également abstraite
  - Utilisé lorsqu'il n'est pas logique qu'une classe de base ait une implémentation
    * Mais les classes concrètes doivent le mettre en œuvre

```
virtual void draw() = 0        // Shape
virtual void defend() = 0      // Shape
```

Fonctions virtuelles pures et classes abstraites




```
class Shape {                    // Abstract
private:
       // attributes common to all shapes
public:
      virtual void draw() = 0;      // pure virtual function
      virtual void rotate() = 0;    // pure virtual function
      virtual ~Shape();
      . . .
};
```

```
class Circle: public Shape {                // Concrete
private:
      // attributes for a circle
public:
      virtual void draw() override {
          // code to draw a circle
       }
       virtual void rotate() override {
          // code to rotate a circle
       }
       virtual ~Circle();
       . . .
};
```

Classe de base abstraite

- Le résumé soit instancié

```
Shape shape;                    // Error
Shape *ptr = new Shape();       // Error
```

- Nous pouvons utiliser des pointeurs et des références pour faire référence dynamiquement à des classes concrètes dérivées de alors

```
Shape *ptr = new Circle();
prt->draw();
ptr->rotate();
```

Classes abstraites comme interfaces

Qu'est-ce qu'utiliser une classe comme interface ?

- Une classe abstraite qui n'a que des fonctions virtuelles pures
- Ces fonctions fournissent un ensemble général de services à l'utilisateur de la classe
- Fourni en tant que public
- Chaque sous-classe est libre d'implémenter ces services selon ses besoins
- Chaque service (méthode) doit être implémenté
- Les informations sur le type de service sont strictement appliquées

Un exemple imprimable + `C++` ne fournit pas de vraies interfaces + Nous utilisons des classes abstraites et des fonctions virtuelles pures pour y parvenir. + Supposons que nous voulions pouvoir fournir un support « imprimable » pour n'importe quel objet de notre choix sans connaître son implémentation au moment de la compilation.

```
std :: cout << any_object << std :: endl;
```

- `any_object` doit être conforme à l'interface `Printable`

An Printable example

```
class Printable {
   friend ostream &operator<<(ostream &, const Printable &obj);
public:
   virtual void print(ostream &os) const = 0;
   virtual ~Printable() {};
   . . .
};

ostream &operator<<(ostream &os, const Printable &obj) {
   obj.print(os);
   return os;
}
```

Un exemple de formes

```
class Any_Class : public Printable {
public:
   // must override Printable::print()
   virtual void print(ostream &os) override {
      os << "Hi from Any_Class" ;
   }
   . . .
};

class Any_Class : public Printable {
public:
   // must override Printable::print()
   virtual void print(ostream &os) override {
      os << "Hi from Any_Class" ;
   }
   . . .
};
Any_Class *ptr= new Any_Class();
cout << *ptr << endl;

void function1 (Any_Class &obj) {
   cout << obj << endl;
}

void function2 (Printable &obj) {
   cout << obj << endl;
}
function1(*ptr);          // "Hi from Any_Class"
function2(*ptr);          // "Hi from Any_Class"
```

```
class Shape {
public:
   virtual void draw() = 0;
   virtual void rotate() = 0;
   virtual ~Shape() {};
   . . .
};
```

```
class Circle : public Shape {
public:
   virtual void draw() override { /* code */ };
   virtual void rotate() override { /* code */ };
   virtual ~Circle() {};
   . . .
};
```

```
class I_Shape {
public:
   virtual void draw() = 0;
   virtual void rotate() = 0;
   virtual ~I_Shape() {};
   . . .
};
```

```
class Circle : public I_Shape {
public:
   virtual void draw() override   { /* code */ };
   virtual void rotate() override { /* code */ };
   virtual ~Circle() {};
   . . .
};
```

```
vector< I_Shape *> shapes;

I_Shape *p1 = new Circle();
I_Shape *p2 = new Line();
I_Shape *p3 = new Square();

for (auto const &shape: shapes) {
   shape->rotate();
   shape->draw();
}
// delete the pointers
```